Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Distributed CDN Simulator with Latency-Aware Routing and Adaptive Caching

Submitted by

Sr. No.	Student Name	PRN
1	More Geet Dattatraya	1262250006
2	Shinde Soham Manoj	1262250009
3	Jagtap Prasad Sachin	1262250018
4	Gaikwad Shreyash Sachin	1262250005

Group No. 2 Panel E Batch E-1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Swati

Academic Year 2026–2027

Submission Date 21st Oct 2026



Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "Distributed CDN Simulator with LatencyAware Routing and Adaptive Caching" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel E, Batch E-1, Group No. 2, under my guidance during the academic year 2026–2027.

Student Name	PRN
More Geet Dattatraya	1262250006
Shinde Soham Manoj	1262250009
Jagtap Prasad Sachin	1262250018
Gaikwad Shreyash Sachin	1262250005

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Dr. Swati

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We thank Dr. Swati, our project guide and course faculty for the Computer Networks Laboratory (CSE20110), for the guidance provided during this mini-project. We are grateful to the Department of Computer Engineering and Technology, MIT World Peace University, Pune, for the laboratory course and resources that made this work possible. We also acknowledge the

open-source projects on which the implementation is built, including Python, FastAPI, React, Vite, Recharts and SQLite, and the IETF specifications that describe the protocols studied in this project.

Group No. 2

Abstract

Content Delivery Networks reduce latency and origin load by serving content from edge servers close to users, but choosing the right edge requires balancing network latency, server load, availability and cache contents. This project implements a working miniature CDN to study these trade-offs. An origin server and three independent edge-server processes communicate over HTTP/1.1 on TCP; clients locate an edge through a custom DNS-inspired resolver protocol over UDP. Six routing strategies (random, nearest replica, lowest latency, least loaded, a weighted hybrid of latency, load and packet loss, and health-aware failover) and four cache configurations (LRU, LFU, TTL and disabled) are implemented. A FastAPI control plane performs active and passive health checking, runs reproducible experiments, stores results in SQLite and streams live metrics over WebSocket to a React dashboard. 89 automated tests pass. In Local Mode experiments with simulated link profiles, hybrid routing achieved the lowest average latency (147 ms versus 436 ms for nearest replica) under concurrent load, caching cut origin fetches by about 40 %, and a failed edge was detected within milliseconds by passive checks. Mininet execution remains future work.

Keywords: content delivery network, edge caching, latency-aware routing, load balancing, HTTP/1.1, UDP name resolution, LRU, LFU

Table of Contents

Acknowledgement 2

Abstract 3

Table of Contents 3

List of Figures 5

List of Tables 5

1 Introduction and Project Objectives 6
1.1 Introduction and Background 6
1.2 Problem Statement 7
1.3 Scope 7
1.4 Objectives 8
2 Proposed Design and Requirements 10
2.1 Architecture and Network Topology 10
2.2 Tools and System Requirements 11
2.3 Addressing and Protocol Selection 12
3 Implementation and Technical Working 14
3.1 Methodology 14
3.2 Configuration or Code Implementation 15
3.3 Protocols and Technical Working 16
3.4 Execution Procedure 18
4 Testing and Results 19
4.1 Test Plan 19
4.2 Results and Execution Evidence 22
4.3 Objective Achievement 25
5 Discussion and Conclusion 26
5.1 Discussion and Comparative Analysis 26
5.2 Advantages and Applications 26
5.3 Limitations 27
5.4 Challenges and Troubleshooting 27
5.5 Future Scope 28
5.6 Conclusion 28
References 30

AI Usage Declaration 30

Appendices 32

Appendix A Code and Configuration 32

Appendix B Additional Evidence 32

Appendix C Individual Contributions 34

List of Figures

Figure 2.1: Architecture of the Distributed CDN Simulator 10

Figure 3.1: Request lifecycle across resolver, edge, cache and origin 15

Figure 3.2: Dashboard overview page of the running system (live metrics) 19

Figure 3.3: Edge Servers page showing per-edge health, measured RTT, load and cache 19

Figure 4.1: Latency and throughput by routing strategy (Experiment A) 22

Figure 4.2: Origin fetches and hit ratio by cache policy (Experiment B) 23

Figure 4.3: Failure experiment results in the dashboard (per-request resolution) 24

Figure 4.4: Requests per edge during the failure, per-request versus DNS-style resolution .. 25

Figure 4.5: Average latency over time after link degradation (Experiment D) 25

Figure B.1: Routing comparison results as displayed in the Routing Lab 34

Figure B.2: Network Topology page (conceptual topology in Local Mode) 34

List of Tables

Table 1.1: Project objectives and verification methods 9

Table 2.1: Tools, verified versions and purpose 11

Table 2.2: Host addresses, ports and roles 12

Table 2.3: CNRP message formats 13

Table 3.1: Principal backend modules 15

Table 3.2: Implemented routing strategies 17

Table 4.1: Functional test results 20

Table 4.2: Routing comparison, 400 requests per strategy, concurrency 24, mean of 3 repetitions 22

Table 4.3: Cache comparison, 400 requests per policy, 2 MB per edge, mean of 3 repetitions 23

Table 4.4: India edge failed at t = 3 s and restored at t = 7 s (600 requests at 60 req/s, nearest routing) ..... 24 **Table 4.5:** Objective achievement 25

Table 5.1: Issues encountered and their resolution 28

Table A.1: Important files 33

Table B.1: Automated test results 33

Table C.1: Individual contributions 35

1 Introduction and Project Objectives

1.1 Introduction and Background

A Content Delivery Network (CDN) is a distributed system of servers that delivers web content on behalf of an **origin server**, the authoritative source of the content. Copies of popular objects are kept in the caches of **edge servers** placed closer to users, so that most requests are answered without crossing long network paths to the origin. Large commercial CDNs operate hundreds of thousands of such servers and combine request routing, caching and failure handling to meet performance targets [6].

Three quantities dominate the user-visible performance of a CDN. **Latency** is the time between issuing a request and receiving the last byte of the response; it grows with the roundtrip time of the path, with transmission time for large objects, and with retransmissions caused by packet loss [8]. **Server load** matters because an edge that is already serving many requests queues new ones, so a nearby but overloaded server can respond more slowly than a distant idle one. **Caching** determines how often an edge must fetch an object from the origin: web request

popularity is strongly skewed and approximately follows a Zipf-like distribution, which is why small caches can absorb a large fraction of requests [7].

Clients are usually directed to an edge by the Domain Name System: a resolver answers a query for the CDN hostname with the address of a selected edge, and the answer is cached for its time-to-live [3]. Content is then transferred with HTTP over TCP [1], [2], [5]. Intelligent routing therefore happens at resolution time, and it must use measurements such as latency, load and health to pick an edge. Content-delivery efficiency is the combined result of these choices: lower latency for users, fewer and smaller transfers from the origin, and continued service when an edge fails.

This project builds a small but genuinely networked CDN in which these mechanisms can be observed and compared. All communication uses real sockets: HTTP/1.1 over TCP for content and a DNS-inspired binary protocol over UDP for resolution. Experiments are repeatable and their results are measured from real requests, which makes the system suitable for laboratory study of routing and caching trade-offs.

1.2 Problem Statement

Given several edge servers with different network distance, bandwidth, packet loss, capacity and cache contents, a CDN must decide, for each client request, which edge should serve it and what each edge should keep in its limited cache. A purely distance-based choice ignores load and can overload the closest edge; a purely load-based choice can send users to distant servers; neither reacts correctly to failures unless server health is tracked. The problem addressed by this project is to implement and evaluate an edge-selection and caching system that considers latency, load, packet loss and availability, and to quantify the effect of each strategy on latency, throughput, cache-hit ratio, origin traffic and failure recovery under controlled, repeatable conditions.

1.3 Scope

The system is intended for students and engineers studying content delivery. Its boundaries are:

- **Included:** one origin server, three edge servers (India, Singapore and USA, all simulated locations), a UDP resolver, six routing strategies, LRU/LFU/TTL caching, health checking with failure injection, a traffic generator, experiment management with SQLite persistence, a REST/WebSocket control plane and a web dashboard.

- **Local Mode (implemented and tested):** all processes run on one computer over the loopback interface. Geographic distance and link conditions are simulated inside each edge by delaying responses according to a configurable profile (base latency, jitter, bandwidth and packet loss).
- **Mininet Mode (implemented, not executed):** a topology script places the same processes on Mininet hosts connected by links shaped with tc/netem [9]. It requires Linux and was not run on the Windows development machine.
- **Assumptions:** a single client region, a static content catalogue of 32 objects (about 9 MB), in-memory caches, and one experiment at a time.
- **Out of scope:** real geographic deployment, standards-compliant DNS, HTTPS, HTTP/2 or HTTP/3, cache revalidation, and production-scale load.

1.4 Objectives

The objectives below are stated so that each can be checked through implementation, automated tests or experiments (Table 1.1).

Table 1.1: Project objectives and verification methods

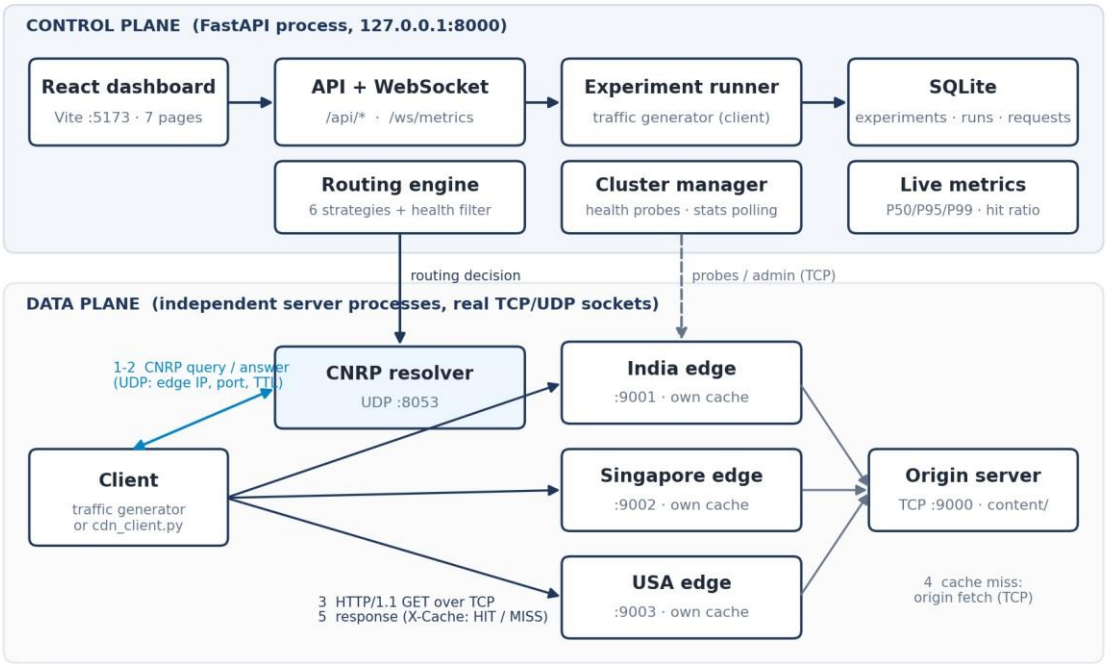
Objective ID	Objective	Verification Method
O1	Serve real objects from an origin through three independent edge servers over HTTP/1.1 on TCP, with correct handling of cache hits, misses, missing objects and malformed requests.	Automated networking and integration tests; commandline download with hash comparison
O2	Resolve the CDN hostname to an edge through a documented UDP protocol with request identifiers, TTL and error codes.	UDP resolver tests (success, unknown name, malformed input, timeout)
O3	Implement at least four routing strategies, including a configurable hybrid latency/load/loss score, that never select an unhealthy edge when a healthy one exists.	Routing unit tests; routing comparison experiment (A)
O4	Implement LRU, LFU and TTL caches per edge with byte-capacity enforcement, expiry, invalidation and request coalescing.	Caching unit tests; cache comparison experiment (B)
O5	Detect a failed edge, reroute new requests to healthy edges and measure detection and recovery time.	Failure tests; server-failure experiment (C)

O6	Persist experiments and expose live metrics through a REST/WebSocket control plane and dashboard.	API, persistence and WebSocket tests; frontend tests; scripted browser checks
O7	Run the CDN on an emulated network topology with shaped links (Mininet).	Mininet smoke test

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

The system is divided into a **data plane**, the processes that carry content bytes, and a **control plane**, which configures, observes and experiments on the data plane (Figure 2.1). Every arrow in the figure is a real network connection; no component calls another through inmemory shortcuts.



Local Mode shown (all processes on 127.0.0.1). Mininet Mode runs the same processes on Mininet hosts with tc/netem-shaped links (script implemented, not executed).

Figure 2.1: Architecture of the Distributed CDN Simulator

Data plane. The origin server (TCP port 9000) holds the content catalogue. Each edge server is a separate operating-system process with its own cache, counters, concurrency limit and simulated network profile, and has two TCP listeners: a data port (9001–9003) for client requests and health probes, and an admin port (9101–9103) through which the control plane manages it. A client first sends a resolution query to the CNRP resolver (UDP port 8053), receives the address of the selected edge, and then downloads the object with HTTP/1.1. On a cache miss the edge fetches the object from the origin.

Control plane. A FastAPI [11] application (TCP port 8000) contains the routing engine, a cluster manager that probes edge health and polls statistics, the experiment runner and traffic generator, a live metrics window and the SQLite database. The React dashboard (served by Vite on port 5173) uses the REST API and a WebSocket stream [12]. In Local Mode all

processes listen on 127.0.0.1; the Mininet topology places them on separate hosts of an emulated network (Section 2.3).

2.2 Tools and System Requirements

Table 2.1 lists the software and hardware actually used. Versions were read from the dependency manifests (`backend/requirements.txt`, `frontend/package.json`) and from the development environment.

Table 2.1: Tools, verified versions and purpose

Tool or Resource	Version or Specification	Purpose
Operating system	Windows 11 Home (10.0.26200)	Development and Local Mode execution
Hardware	AMD Ryzen 9 8945HS (8 cores / 16 threads), 15.3 GB RAM	All processes run on this machine
Python	3.13.2	Backend language (asyncio sockets)
FastAPI / Uvicorn	0.115.6 / 0.32.1	Control-plane REST API and ASGI server
Pydantic	2.10.4	Request validation
websockets	13.1	WebSocket transport for Uvicorn
SQLite (Python sqlite3)	3.45.3	Persistence of experiments, runs and requests
psutil	6.1.1	Process CPU and memory figures; parent watchdog
pytest / pytest-asyncio / httpx	8.3.4 / 0.25.0 / 0.28.1	Backend automated tests
Node.js / npm	20.19.4 / 11.7.0	Frontend tool chain
React / React Router	18.3.1 / 6.28.0	Dashboard user interface
TypeScript / Vite	5.6.3 / 5.4.11	Typed frontend source and build tool
Recharts / Lucide React	2.13.3 / 0.460.0	Charts and icons

Vitest / Testing Library	2.1.8 / 16.1.0	Frontend automated tests
Google Chrome	155.0.8059.85	Scripted dashboard checks and screenshots (DevTools Protocol)
Mininet, Open vSwitch, Wireshark	Not installed	Documented for Linux; not used in this environment

2.3 Addressing and Protocol Selection

This is a socket-programming project, so addressing is expressed as host addresses and ports (Table 2.2). In Local Mode every service shares the loopback interface, so no gateway is involved; the Mininet addresses are those configured in `mininet/topology.py` and were not executed.

Table 2.2: Host addresses, ports and roles

Device / Interface	IP / Prefix	Gateway	Role
Control plane (REST + WebSocket)	127.0.0.1, TCP 8000	N/A (loopback)	API, health manager, experiments
CNRP resolver	127.0.0.1, UDP 8053	N/A (loopback)	Edge selection for cdn.sim.local
Origin server	127.0.0.1, TCP 9000	N/A (loopback)	Authoritative content source
India edge	127.0.0.1, TCP 9001 / 9101	N/A (loopback)	Edge data port / admin port
Singapore edge	127.0.0.1, TCP 9002 / 9102	N/A (loopback)	Edge data port / admin port
USA edge	127.0.0.1, TCP 9003 / 9103	N/A (loopback)	Edge data port / admin port
Dashboard (Vite)	127.0.0.1, TCP 5173	N/A (loopback)	Web UI; proxies /api and /ws
Mininet hosts (not executed)	client 10.0.0.1/24, origin 10.0.0.2/24, edges 10.0.0.11–13/24	N/A (single L2 switch)	Emulated topology with tc/netem links

TCP and HTTP/1.1 for content. Content transfer requires reliable, ordered delivery, so it uses TCP [5] with HTTP/1.1 message framing [2]. The project contains its own strict HTTP/1.1 implementation on asyncio streams [10] so that every byte on the wire is produced by the project and can be inspected. Requests are framed by `Content-Length` only (`Transfer-Encoding` is rejected with 501), the request head is limited to 16 KiB (431 otherwise), malformed request lines, header folding, missing `Host` and conflicting lengths return 400, and unsupported versions return 505. Every response carries `Content-Length`. The client opens one TCP connection per request, which makes each transfer visible as a complete handshake, exchange and teardown.

UDP for resolution. Name resolution is a short request/response exchange that tolerates loss through retransmission, so it uses UDP [4], following the DNS model [3]. The CDN Name Resolution Protocol (CNRP) is DNS-inspired but deliberately simpler and is **not** standardscompliant DNS. Its fixed binary layout is summarised in Table 2.3; all integers are in network byte order.

Table 2.3: CNRP message formats

Message	Fields (bytes)	Notes
Query	magic `CNRQ` (4), version (1), flags (1), request id (2), name length (1), hostname (N)	Flag bit 0 requests a routing explanation
Answer	magic `CNRR` (4), version (1), rcode (1), request id (2), TTL (4), IPv4 (4), port (2), edge-id length (1) + id, explanation length (2) + text	rcode: 0 OK, 1 FORMERR, 2 SERVFAIL, 3 NXDOMAIN, 4 BADVERS

`cdn.sim.local` is answered by the active routing strategy, `<edge>.cdn.sim.local` pins one edge, and any other name returns NXDOMAIN. Datagrams without the `CNRQ` magic are dropped silently. With DNS-style resolution the client caches the answer for its TTL (5 s), so routing happens at resolution time; a separate, explicitly non-DNS per-request mode resolves before every request and is used for routing comparisons.

3 Implementation and Technical Working

3.1 Methodology

The implementation followed an incremental, test-driven sequence in which each layer was verified over real sockets before the next layer was built:

1. **Environment inspection** — operating system, Python, Node.js and port availability were checked; Local Mode was chosen as the primary target because Mininet requires Linux.
2. **Data plane** — HTTP/1.1 server and client, origin server, edge server with simulated network profile, and the caching engine; verified by downloading real files and observing MISS followed by HIT.
3. **Networking and routing** — CNRP protocol, resolver, routing strategies and health checking with failure injection; verified with UDP and failover tests.
4. **Control plane** — FastAPI endpoints, WebSocket stream, SQLite persistence and the experiment runner.
5. **Frontend** — seven dashboard pages connected only to real backend endpoints.
6. **Experiments and debugging** — routing, caching, failure and degradation experiments; issues found during testing were diagnosed and fixed (Section 5.4).
7. **Documentation** — architecture, API, experiment, Mininet and Wireshark guides.

Figure 3.1 shows the message sequence for one client request, which is the central workflow of the system.

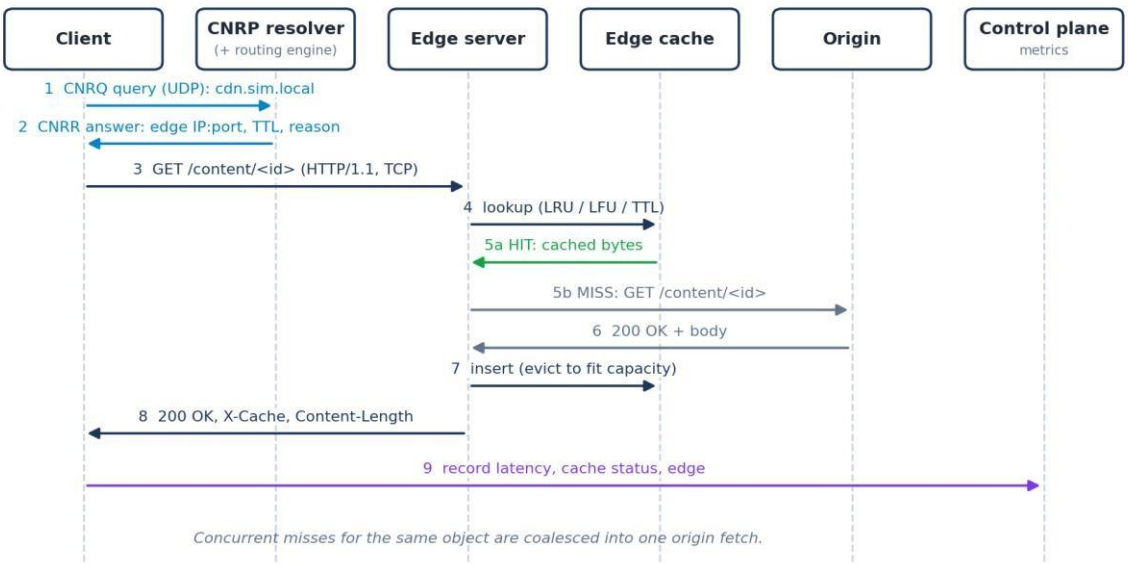


Figure 3.1: Request lifecycle across resolver, edge, cache and origin

3.2 Configuration or Code Implementation

The backend is organised into packages with a single responsibility each; the main modules are listed in Table 3.1.

Table 3.1: Principal backend modules

Module (backend/app/)	Responsibility
networking/http.py	Strict HTTP/1.1 server and client on asyncio TCP streams; crash simulation via stop()
networking/resolver.py	CNRP encoding/decoding, UDP resolver server and stubresolver client
origin/server.py	Origin server: catalogue, safe file access, configurable response delay
edge/server.py, edge/profile.py	Edge server (data + admin listeners) and simulated link profile
caching/policies.py, caching/edge_cache.py	LRU, LFU, TTL policies; per-edge cache with coalescing
routing/strategies.py, routing/engine.py	Six strategies, normalisation, health filtering, decision log
core/cluster.py	Active/passive health checks, statistics polling, routing inputs
core/launcher.py, core/services.py	Process launcher (one OS process per server) and composition root
traffic/generator.py, experiments/runner.py	Seeded workloads and experiment orchestration
persistence/db.py, api/routes.py, api/ws.py	SQLite layer, REST endpoints, WebSocket metrics

Hybrid routing. For each healthy edge the hybrid strategy min-max normalises the measured latency, load and estimated packet loss across the candidates (a missing value counts as worst, identical values as zero) and selects the edge with the lowest weighted score; weights must be non-negative and are normalised to sum to one. Ties are broken deterministically by configured priority and edge identifier:

```
def scores(self, edges):     nl = normalize({e.edge_id:
e.latency_ms for e in edges})     nd = normalize({e.edge_id:
e.load for e in edges})     np_ = normalize({e.edge_id:
e.packet_loss for e in edges})
    w = self.weights                      # e.g. latency 0.5, load 0.4, loss 0.1
return {e.edge_id: w["latency"] * nl[e.edge_id] + w["load"] * nd[e.edge_id]
                       + w["packet_loss"] * np_[e.edge_id] for e in edges}
```

Request coalescing. When several requests miss on the same object at the same time, only the first fetches it from the origin; the others wait on the same future and are reported as `COALESCED`:

```
pending = self._inflight.get(key)
if pending is not None:                      # identical fetch already in flight
self.coalesced += 1
    return await asyncio.shield(pending), "COALESCED" self.misses
+= 1
fut = asyncio.get_running_loop().create_future()
self._inflight[key] = fut                    # followers will await this future
```

Simulated link profile (Local Mode). Each edge delays its response by a propagation delay (base latency ± jitter), a transmission delay (object size ÷ bandwidth) and a loss penalty: the number of lost segments is drawn from a binomial distribution over ceil(size ÷ 1460) + 1 segments, and each lost segment adds max(2 × base latency, 10 ms). These delays are labelled as simulated throughout the API and dashboard.

3.3 Protocols and Technical Working

The implemented request lifecycle (Figure 3.1) is:

1. The client (traffic generator or `scripts/cdn_client.py`) sends a CNRQ query for `cdn.sim.local` over UDP.
2. The resolver asks the routing engine for a decision. Unhealthy edges are removed first; the active strategy selects among the rest and records its reason. The CNRR answer carries the edge address, port, TTL and, if requested, the explanation. With no healthy edge the answer is SERVFAIL.
3. The client opens a TCP connection to the edge's data port and sends `GET /content/<object_id>`.
4. The edge validates the identifier (1–5 path segments of `[A-Za-z0-9._-]`, no segment starting with a dot, no percent-encoding), waits for one of its concurrency slots, and looks the object up in its own cache.
5. On a **hit** the cached bytes are used. Expired TTL entries are never served.
6. On a **miss** the edge requests the object from the origin over a new TCP connection (unless an identical fetch is already in flight). A missing object yields 404 and an unreachable origin 502.
7. The object is inserted according to the active policy, evicting entries until it fits; objects larger than the capacity are served but not cached. The response is sent with `X-Cache: HIT|MISS|COALESCED|BYPASS`.
8. The client records the measured latency, status, edge and cache status; the control plane aggregates these records into live metrics and, for experiments, stores them in SQLite.

Table 3.2: Implemented routing strategies

Strategy	Selection rule	Input
Random	Uniform choice among healthy edges (seeded for reproducibility)	—
Nearest replica	Smallest configured distance from the client region	Configured
Lowest latency	Lowest EWMA of measured health-probe round-trip time	Measured
Least loaded	Lowest max(active + queued, client in-flight) ÷ concurrency limit	Measured
Hybrid	Lowest weighted score of normalised latency, load and packet loss	Measured + estimated
Health-aware failover	Highest-priority healthy edge (primary/backup)	Configured + health

Caching policies. LRU evicts the least recently used object (ordered dictionary, O(1)); LFU evicts the least frequently used object with ties broken by recency (frequency buckets, O(1)); TTL expires objects a fixed time after insertion and, under capacity pressure, evicts the one closest to expiry. All policies enforce a byte capacity that is never exceeded, report every eviction with its reason, and can be switched at run time (the cache then restarts empty). Hit ratio is defined as hits ÷ (hits + misses + coalesced).

Health checking and failure handling. The cluster manager probes `GET /_health` on every edge's data port every 0.5 s with a 0.8 s timeout; two consecutive failures mark an edge unhealthy and two successes mark it healthy. In addition, three consecutive client transport failures (refused, reset or timeout) mark an edge unhealthy immediately (passive detection). A failure is injected by closing the edge's data listener and aborting its connections, which emulates a crashed process while its admin port remains available for restore.

Connection handling and errors. All socket operations have timeouts; the HTTP server supports persistent connections with a 15 s idle timeout; handler exceptions become 500 responses without tracebacks; client errors are classified as connect, timeout, reset or protocol failures. The control plane returns JSON errors of the form `{"error", "detail"}` with status 400, 404, 409, 422 or 502.

3.4 Execution Procedure

Prerequisites are Python 3.12 or newer, Node.js 18 or newer and the free ports listed in Table 2.2. The following commands were executed successfully on the development machine (Windows paths shown; on Linux replace `.venv\Scripts\python` with `.venv/bin/python`).

Backend (starts the control plane, which launches the origin and the three edge processes):

```
cd backend python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Frontend (second terminal), then open http://127.0.0.1:5173 in a browser:

```
cd frontend npm
install npm run
dev
```

Command-line client (UDP resolution followed by an HTTP download) and **tests**:

```
cd backend
.venv\Scripts\python scripts\cdn_client.py images/photo-05.png --repeat 2
.venv\Scripts\python -m
pytest cd ..\frontend npm
test npm run build
```

Experiments are started from the dashboard (Traffic Generator, Routing Lab, Cache Lab or Experiments page) or through `POST /api/experiments`; results can be exported as CSV. Mininet Mode requires Ubuntu with Mininet, Open vSwitch and root privileges and is started with `sudo python3 mininet/topology.py`; this procedure is documented in `mininet/README.md` but was not executed.

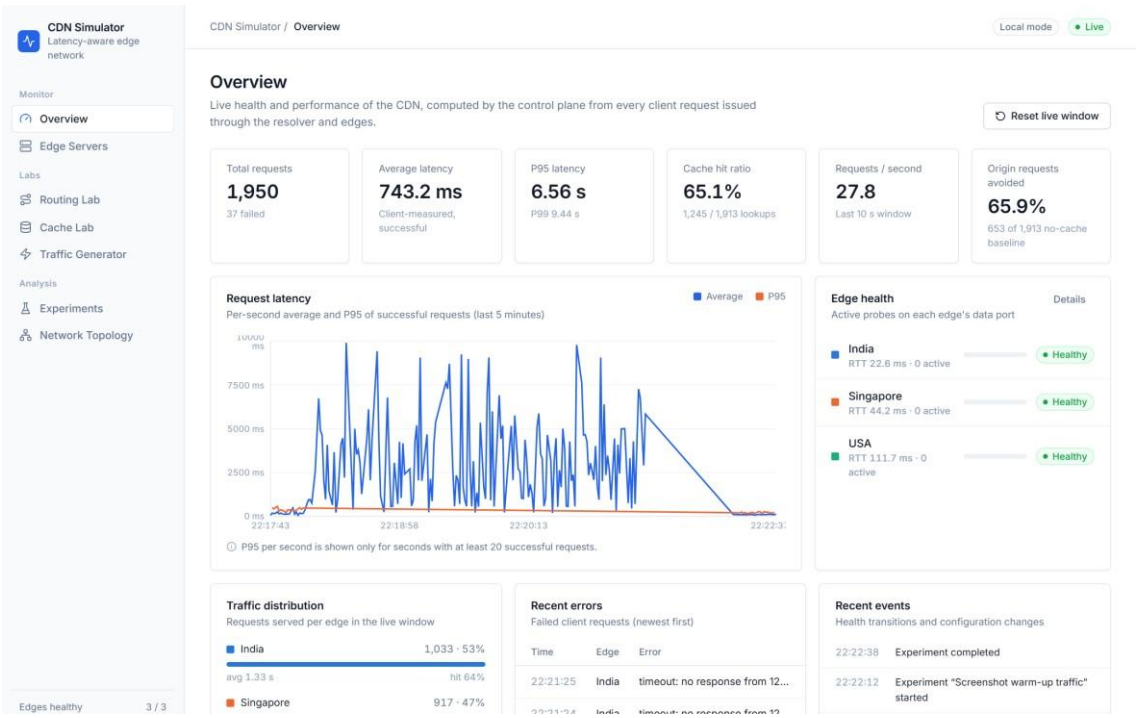


Figure 3.2: Dashboard overview page of the running system (live metrics)

Figure 3.2 is a screenshot of the running dashboard. All values come from the control plane: request counts, latency percentiles, cache-hit ratio and origin requests avoided are computed from client-side records of real requests, and the edge-health panel reflects active probe results.

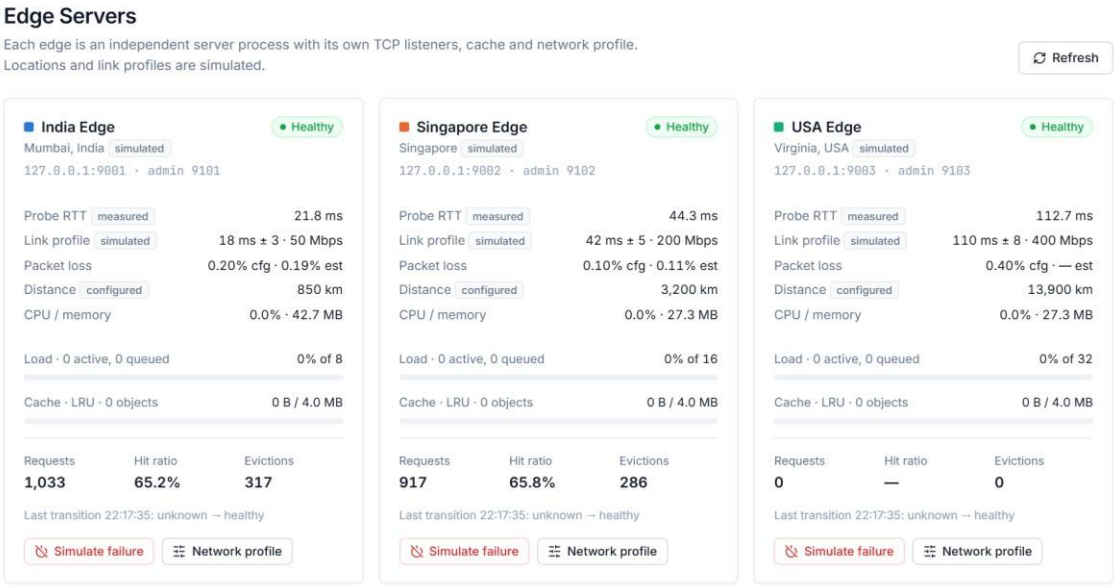


Figure 3.3: Edge Servers page showing per-edge health, measured RTT, load and cache

Figure 3.3 shows the three edge processes with their measured probe round-trip times next to the simulated profile values, current load against each edge's concurrency limit, cache occupancy and the failure-simulation control.

4 Testing and Results

4.1 Test Plan

Testing combined three kinds of evidence. (1) **Automated tests:** 79 backend tests (pytest) exercise routing, caching, HTTP and UDP networking against servers on ephemeral loopback ports, the REST API, persistence and the WebSocket stream; 10 frontend tests (Vitest) cover WebSocket reconnection, API error handling and interface states. (2) **Scripted browser checks:** the running dashboard was driven through the Chrome DevTools Protocol (16 checks) to confirm that every page renders without console errors and that controls change backend state. (3) **Experiments:** four experiment types were run in Local Mode on the hardware in Table 2.1.

The success criterion for a functional test is that the observed behaviour matches the expected behaviour exactly (status codes, cache status, selected edge, persisted data). Normal operation

and failure or invalid-input cases were both tested. Table 4.1 summarises representative tests; the full list of passing test names is in `submission/evidence/backend_pytest.txt` and `frontend_vitest.txt`.

Table 4.1: Functional test results

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	GET an image through an edge twice (empty cache)	200 with identical bytes; MISS then HIT; one origin fetch	As expected; the origin's request counter increased by exactly one	Pass
T2	cdn_client.py images/photo-05.png -repeat 2	Valid PNG; MISS then HIT; same hash	MISS 53.8 ms, HIT 24.2 ms, identical SHA-256, valid 112×112 PNG	Pass
T3	Missing object; traversal ids (`../`, `%2e%2e`, `.hidden`)	404 for missing; 400 for invalid ids	As expected at both edge and origin	Pass
T4	8 malformed requests and a 20 KB header	400 / 405 / 501 / 505; 431 for oversize head	All status codes matched	Pass
T5	20 concurrent misses for one object	One origin fetch; 1 MISS + 19 COALESCED	Exactly one fetch; statuses as expected	Pass

Test ID	Test and Input	Expected Result	Observed Result	Status
T6	UDP query for cdn.sim.local; unknown name	OK answer with edge, port, TTL; NXDOMAIN	As expected; explanation only with EXPLAIN flag	Pass
T7	Malformed CNRP datagrams; no server	FORMERR / BADVERS; garbage dropped; client timeout	As expected (3 FORMERR/BADVERS, 1 dropped)	Pass
T8	LRU and LFU eviction sequences	LRU evicts least recent; LFU least frequent, ties by recency	Eviction order matched in every case	Pass

T9	TTL expiry with controlled clock	Expired entries never served; purged before insert	As expected	Pass
T10	500 random insertions per policy; 1001 B object in 1000 B cache	Capacity never exceeded; oversize rejected	Invariant held for LRU, LFU and TTL	Pass
T11	Each routing strategy on a fixed fleet; two unhealthy edges	Expected edge; unhealthy edges excluded	All six strategies chose the healthy edge in 30 trials each	Pass
T12	Simulate failure of the nearest edge via API	Edge marked unhealthy; requests go to next edge	Next request served by Singapore with 'excluded unhealthy' reason; restored afterwards	Pass
T13	Create, start, complete and export a cachecomparison experiment	Results persisted; CSV exported	Runs and 120 request records read back from a new database connection	Pass
T14	Invalid API input (unknown strategy, zero weights, tiny capacity)	422 / 400 with error detail	As expected	Pass
T15	WebSocket subscription during a failure	hello, snapshots, failure event, pong	All messages received	Pass
T16	Dashboard pages and controls (browser automation)	No console errors; controls change backend state	16 of 16 checks passed	Pass
T17	Mininet topology smoke test	Traffic crosses shaped links	Not executed (requires Linux)	Not executed
Test ID	**Test and Input**	**Expected Result**	**Observed Result**	**Status**
T18	Wireshark capture of MISS/HIT exchange	No origin traffic on a HIT	Not executed (Wireshark not installed)	Not executed

4.2 Results and Execution Evidence

All 89 automated tests passed (79 backend, 10 frontend; 0 failures) and the production frontend build succeeded. The experiments below were measured on 20 October 2026 in Local Mode with the default simulated profiles: India 18 ms / 50 Mbit/s / 0.2 % loss / 8 slots, Singapore 42 ms / 200 Mbit/s / 0.1 % / 16 slots, USA 110 ms / 400 Mbit/s / 0.4 % / 32 slots. All runs use a Zipf (s = 1.1) workload over the 32-object catalogue with seed 7 and start with empty caches. The numbers describe behaviour in this simulator on one machine and are not claims about real CDNs.

Experiment A: routing comparison

Table 4.2: Routing comparison, 400 requests per strategy, concurrency 24, mean of 3 repetitions

Strategy	Average	P95	Req/s	Hit ratio	Share % (IN / SG / US)
Random	279.8 ms	1118.2 ms	69.6	56.4 %	37 / 31 / 32
Nearest replica	436.0 ms	748.7 ms	52.6	57.0 %	100 / 0 / 0
Lowest latency	436.2 ms	741.3 ms	52.0	59.2 %	100 / 0 / 0
Least loaded	258.5 ms	1047.7 ms	72.3	58.2 %	12 / 59 / 29
Hybrid	146.9 ms	404.3 ms	142.7	57.2 %	30 / 70 / 0

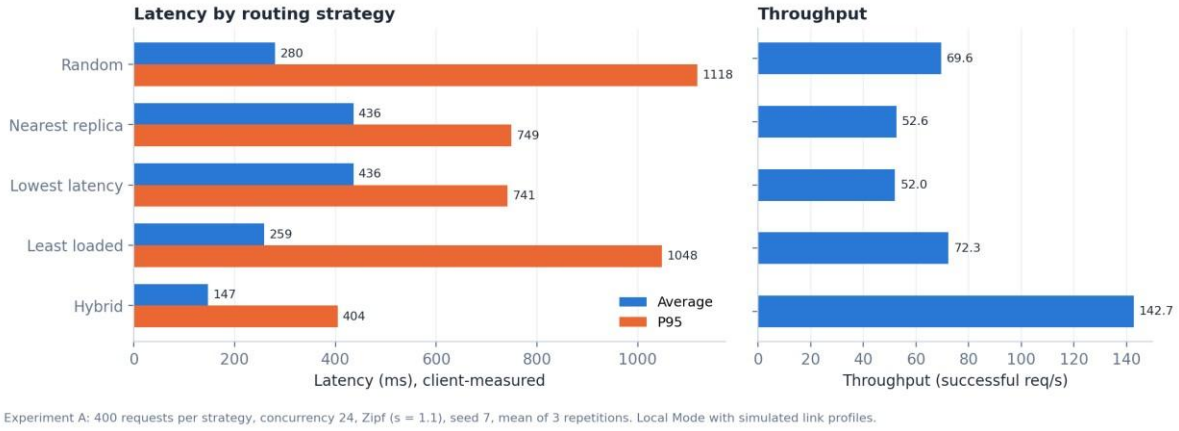


Figure 4.1: Latency and throughput by routing strategy (Experiment A)

Nearest-replica and lowest-latency routing sent every request to the India edge. Its network round trip is the shortest, but with only eight service slots and 50 Mbit/s it queued requests at concurrency 24, giving 436 ms average latency. Least-loaded and random routing spread the load but sent about a third of requests to the 110 ms USA edge, which raised the tail (P95 above one second). Hybrid routing kept traffic on the two nearby edges and achieved the lowest

average (147 ms) and P95 (404 ms) latency and the highest throughput (143 req/s) for this workload.

Experiment B: cache-policy comparison

Table 4.3: Cache comparison, 400 requests per policy, 2 MB per edge, mean of 3 repetitions

Policy	Hit ratio	Origin fetches	Origin bytes	Evictions	Average	P95
Disabled	—	400.0	263.0 MB	0.0	191.6 ms	464.6 ms
LRU	31.1 %	246.3	116.1 MB	242.7	180.3 ms	465.7 ms
LFU	32.8 %	241.0	114.0 MB	237.0	179.4 ms	454.1 ms
TTL (10 s)	30.8 %	245.7	120.4 MB	242.3	181.7 ms	457.8 ms

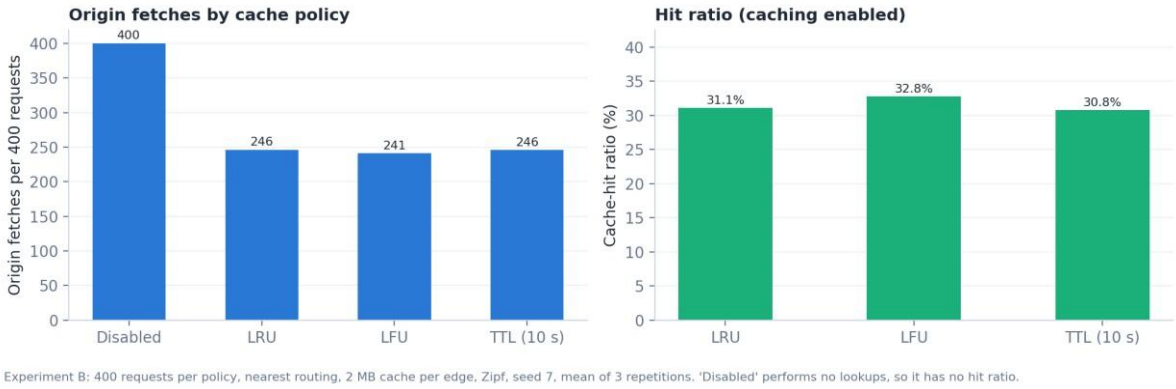


Figure 4.2: Origin fetches and hit ratio by cache policy (Experiment B)

With nearest routing a single edge receives the whole request stream. Every cache policy reduced origin fetches from 400 to about 241–246 and origin bytes by more than half. LFU was marginally best (32.8 % hit ratio), consistent with a skewed popularity distribution [7]; the differences between policies are small because a 2 MB cache can hold only a few of the large objects.

Experiment C: server failure

Table 4.4: India edge failed at t = 3 s and restored at t = 7 s (600 requests at 60 req/s, nearest routing)

Resolution mode	Detection	Detected by	Failed requests	Last failure	Recovery	Failure rate
Per request	2.1 ms	Passive	8 of 249	5.2 ms	882.1 ms	1.33 %
DNS-style (5 s TTL)	5.3 ms	Passive	9 of 250	2043.3 ms	1166.3 ms	1.50 %

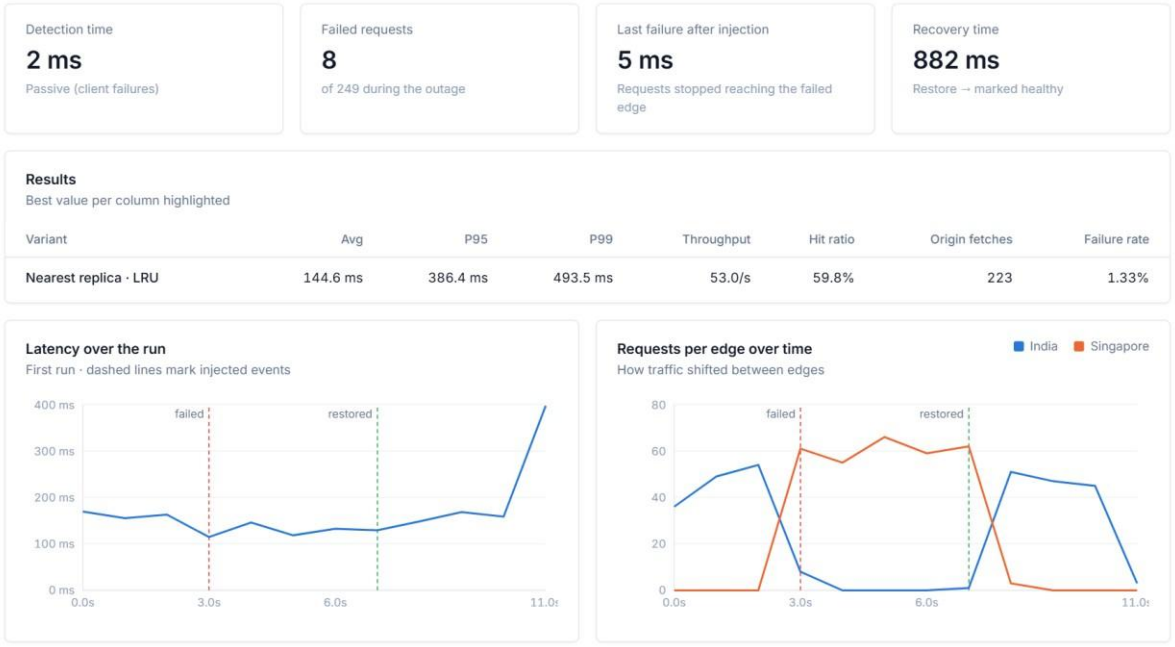


Figure 4.3: Failure experiment results in the dashboard (per-request resolution)

The failed requests were those in flight when the edge crashed; their reset connections triggered passive detection within milliseconds, and the next resolution already excluded India, so traffic moved to Singapore (Figure 4.3, right). After the restore, two successful probes returned India to service. With DNS-style resolution a client that still held a cached India answer kept trying it, and on Windows each refused loopback connection takes about two seconds, so the last failure occurred about 2 s after injection (Figure 4.4). This illustrates that DNS-based routing decides at resolution time, not per request.

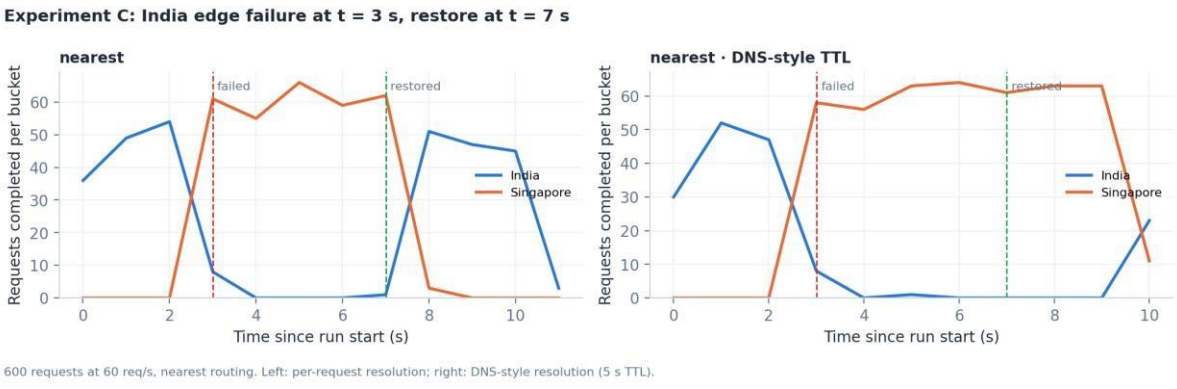


Figure 4.4: Requests per edge during the failure, per-request versus DNS-style resolution

Experiment D: network degradation

At t = 3 s the India link profile was changed to 180 ms base latency and 2 % loss. Lowestlatency routing detected the higher probe RTT and moved traffic to Singapore within about two seconds, finishing with 187.0 ms average latency and no failures. Nearest-replica routing kept

using the degraded link: average latency rose to 2.2 s, throughput fell to 2.7 req/s, 6.2 % of requests exceeded the 10 s client timeout, and the run lasted about 3.4 minutes instead of about 14 seconds (Figure 4.5).

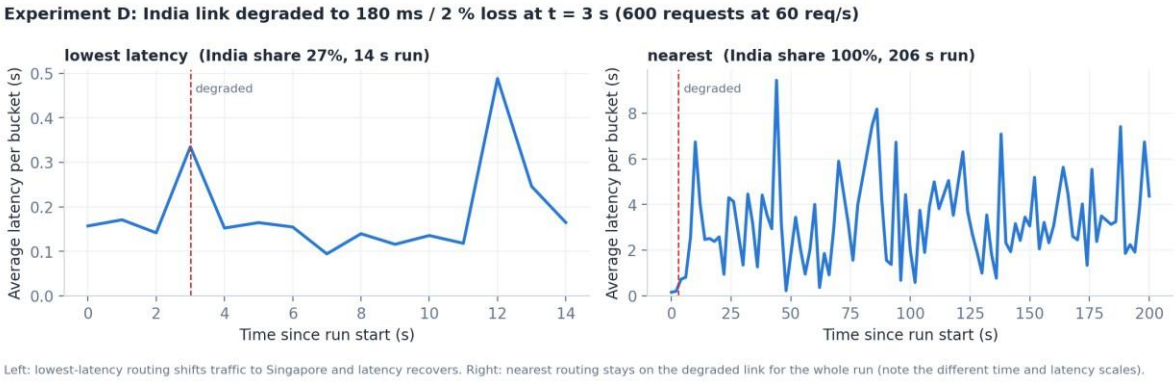


Figure 4.5: Average latency over time after link degradation (Experiment D)

4.3 Objective Achievement

Table 4.5 maps each objective to its evidence. An objective is marked achieved only where tests or experiments demonstrate it.

Table 4.5: Objective achievement

Objective ID	Evidence or Test ID	Outcome
O1	T1–T5; networking and integration tests	Achieved
O2	T6–T7; UDP resolver tests	Achieved
Objective ID	**Evidence or Test ID**	**Outcome**
O3	T11; Experiment A (six strategies compared)	Achieved
O4	T5, T8–T10; Experiment B	Achieved
O5	T12; Experiment C (detection and recovery measured)	Achieved
O6	T13–T16; API, persistence, WebSocket and browser checks	Achieved
O7	T17 (Mininet script exists but was not run)	Not tested

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The experiments show that the best edge depends on more than network distance. When the closest edge has limited capacity, distance- or latency-only routing concentrates load and queuing delay dominates; when load is balanced without regard to latency, distant edges inflate tail latency. The hybrid score balanced both effects and performed best in Experiment A, but this is a result for one workload and one set of weights, not a proof of optimality: at low concurrency, where no queue forms, nearest and lowest-latency routing are expected to perform as well, and different weights shift the balance.

Experiment D shows the complementary weakness of static strategies: nearest-replica routing has no feedback from measurements and continued to use a badly degraded link, while a measurement-driven strategy adapted within seconds. Experiment C shows that passive failure detection from client errors can react much faster than periodic probes, and that DNSstyle caching of answers trades fewer resolutions for slower reaction to failures — the same trade-off that real DNS TTLs impose [3].

For caching, any policy substantially reduced origin traffic compared with no caching, while the differences among LRU, LFU and TTL were small at the tested capacity. Frequencybased eviction suits Zipf-like popularity [7]; recency-based eviction adapts faster to changing popularity, and TTL bounds staleness at the cost of evicting objects that are still popular. Latency gains from caching were modest here because transmission over the simulated India link, rather than the origin, dominated the latency of large objects.

5.2 Advantages and Applications

- **Real protocols:** content and resolution traffic use real TCP and UDP sockets, so behaviour can be inspected with packet-capture tools and the code doubles as a reference for socket programming.
- **Reproducible experiments:** seeded workloads, identical initial cache state for every variant, persisted configuration and CSV export make comparisons repeatable.
- **Transparency:** every routing decision is recorded with its reason and scores, and measured, simulated and estimated values are labelled separately.

- **Applications:** teaching CDN, DNS-based load balancing and caching concepts; prototyping routing heuristics before deploying them; studying failure detection and the effect of DNS TTLs on failover.

5.3 Limitations

- All processes run on one computer over loopback; geographic locations and link conditions are simulated inside each edge, and the measured latencies include these simulated delays.
- The loss model adds a fixed penalty per lost segment in sequence; it is pessimistic for large objects on long, lossy links and does not reproduce TCP congestion control [8].
- CNRP is DNS-inspired and not interoperable with standard DNS resolvers.
- Caches are in memory, TTL is a fixed lifetime rather than HTTP freshness, and conditional requests are not implemented.
- The workload family is limited (Zipf over a 32-object catalogue); experiments A and B used three repetitions and C and D single runs, so small differences are not statistically established.
- Mininet Mode and the Wireshark verification procedure were not executed in this environment.

5.4 Challenges and Troubleshooting

The issues in Table 5.1 were observed during development and testing; each correction was verified by re-running the relevant measurement or test.

Table 5.1: Issues encountered and their resolution

Issue	Cause or Diagnosis	Solution and Verification
Measured probe RTT about 11–14 ms higher than the simulated profile (e.g. 29–31 ms for an 18 ms link)	asyncio sleeps on Windows are quantised to the 15.6 ms system timer; Windows 11 also ignores 1 ms timer requests from windowless child processes	Requested 1 ms timer resolution and opted out of timer power throttling in every server process; RTTs became 21.6 / 46.4 / 114.1 ms for 18 / 42 / 110 ms profiles
Failure detection took 4.3 s	Connecting to a closed loopback port on Windows takes about 2 s, so each probe ran into its timeout	Shorter probe interval and timeout plus passive detection from client failures; detection measured at 2.1 ms in Experiment C
Issue	**Cause or Diagnosis**	**Solution and Verification**

Failure-experiment test reported no detection time	The injection timestamp was taken after the admin call returned, but detection had already happened during the call	Timestamp taken at injection; test passes
'Task was destroyed but it is pending' warnings at test shutdown	Connection-handler tasks outlived the server's stop()	Handlers are tracked and cancelled on stop; test run is clean
Second of two back-toback degradation experiments started on the first one's degraded link and left it degraded	The runner read baseline profiles from a statistics cache not yet refreshed after the previous experiment's restore	Stats refreshed before capturing and after restoring the baseline; a regression test fails without the fix and passes with it; the degradation experiments were rerun
Router warnings found by the scripted browser check	React Router v7 future-flag warnings in the console	Opted in to the future flags; all pages render with zero console messages

5.5 Future Scope

- Execute the Mininet topology on Linux, compare results on real shaped links with Local Mode, and capture the exchanges in Wireshark.
- Replace the simplified loss model with real network emulation, or a throughput model based on TCP behaviour [8].
- Add HTTP freshness and revalidation (`Cache-Control`, `ETag`, conditional GET) [1] and persistent disk caches.
- Learn hybrid weights automatically from measurements, and add multi-region clients and consistent-hashing content placement.
- Run longer, multi-workload experiments with more repetitions and confidence intervals.

5.6 Conclusion

The project delivered a working, networked CDN simulator: an origin, three independent edge processes with their own LRU, LFU or TTL caches, a UDP resolver, six routing strategies with health-aware filtering, active and passive failure detection, a FastAPI control plane with SQLite persistence and a seven-page React dashboard. Objectives O1–O6 were achieved and verified by 89 passing automated tests, scripted browser checks and four experiment types; objective

O7 (Mininet execution) remains untested. The experiments showed, within the simulator, that combining latency and load in routing decisions reduced latency under load, that measurement-driven routing adapts to link degradation while static routing does not, that caching more than halves origin traffic for a Zipf workload, and that passive detection combined with per-request resolution limits the impact of an edge failure.

References

[1] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP Semantics,” IETF, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP/1.1,” IETF, RFC 9112, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9112

[3] P. Mockapetris, “Domain Names – Implementation and Specification,” IETF, RFC 1035, Nov. 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1035

[4] J. Postel, “User Datagram Protocol,” IETF, RFC 768, Aug. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768

[5] W. Eddy, Ed., “Transmission Control Protocol (TCP),” IETF, RFC 9293, Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[6] E. Nygren, R. K. Sitaraman, and J. Sun, “The Akamai network: A platform for highperformance Internet applications,” ACM SIGOPS Operating Systems Review, vol. 44, no. 3, pp. 2–19, Jul. 2010, doi: 10.1145/1842733.1842736.

[7] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and Zipf-like distributions: Evidence and implications,” in Proc. IEEE INFOCOM ’99, New York, NY, USA, 1999, pp. 126–134, doi: 10.1109/INFCOM.1999.749260.

[8] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, “The macroscopic behavior of the TCP congestion avoidance algorithm,” ACM SIGCOMM Computer Communication Review, vol. 27, no. 3, pp. 67–82, Jul. 1997, doi: 10.1145/263932.264023.

[9] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), 2010, doi: 10.1145/1868447.1868466.

[10] Python Software Foundation, “asyncio — Asynchronous I/O,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/asyncio.html (accessed Oct. 20, 2026).

[11] FastAPI, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/ (accessed Oct. 20, 2026).

[12] I. Fette and A. Melnikov, “The WebSocket Protocol,” IETF, RFC 6455, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455

AI Usage Declaration

AI tools were used in this project. Claude Code (Anthropic), an AI coding assistant, was used to generate and debug the source code, automated tests, experiment and figure scripts, the technical documentation and drafts of this report and presentation. The assistant also executed the automated tests and experiments in the development environment; the results reported here are taken from those runs (stored in `submission/evidence/`). The team is responsible for

reviewing the implementation, re-running the tests and experiments, and verifying the content of this report before submission.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	**PRN**	**Signature**
More Geet Dattatraya	1262250006	
Shinde Soham Manoj	1262250009	
Jagtap Prasad Sachin	1262250018	
Gaikwad Shreyash Sachin	1262250005	

Appendices

Appendix A Code and Configuration

The complete source code is in the project repository; key files are listed below. Secrets or credentials are not used by the project.

Table A.1: Important files

Path	Content
backend/app/	Backend packages (networking, routing, caching, origin, edge, core, traffic, experiments, persistence, api, schemas)
backend/app/config.py	Default simulated topology, ports and environment variables
backend/requirements.txt, frontend/package.json	Pinned dependency manifests
backend/tests/	pytest suite: test_routing, test_caching, test_networking, test_metrics, test_api
backend/scripts/cdn_client.py	Command-line CNRP + HTTP client
frontend/src/	React dashboard (pages, components, API client, WebSocket hook)
mininet/topology.py	Mininet topology with tc/netem links (not executed)
docs/	ARCHITECTURE.md, API.md, EXPERIMENTS.md, WIRESHARK.md
scripts/	Report, presentation, figure and screenshot generation; submission validation

Default edge configuration (excerpt from `backend/app/config.py`):

```
{"edge_id": "india-edge", "host": "127.0.0.1", "data_port": 9001, "admin_port": 9101,
 "distance_km": 850, "priority": 1, "concurrency_limit": 8,
  "profile": {"base_latency_ms": 18, "jitter_ms": 3, "bandwidth_mbps": 50, "packet_loss":
0.002}}
```

Appendix B Additional Evidence

Automated test counts per file, from `submission/evidence/backend_pytest.txt`:

Table B.1: Automated test results

Test file	Passed tests
test_api.py	15
test_caching.py	18

test_metrics.py	4
test_networking.py	27
Test file	**Passed tests**
test_routing.py	15
Frontend (src/app.test.tsx)	10
Total	89

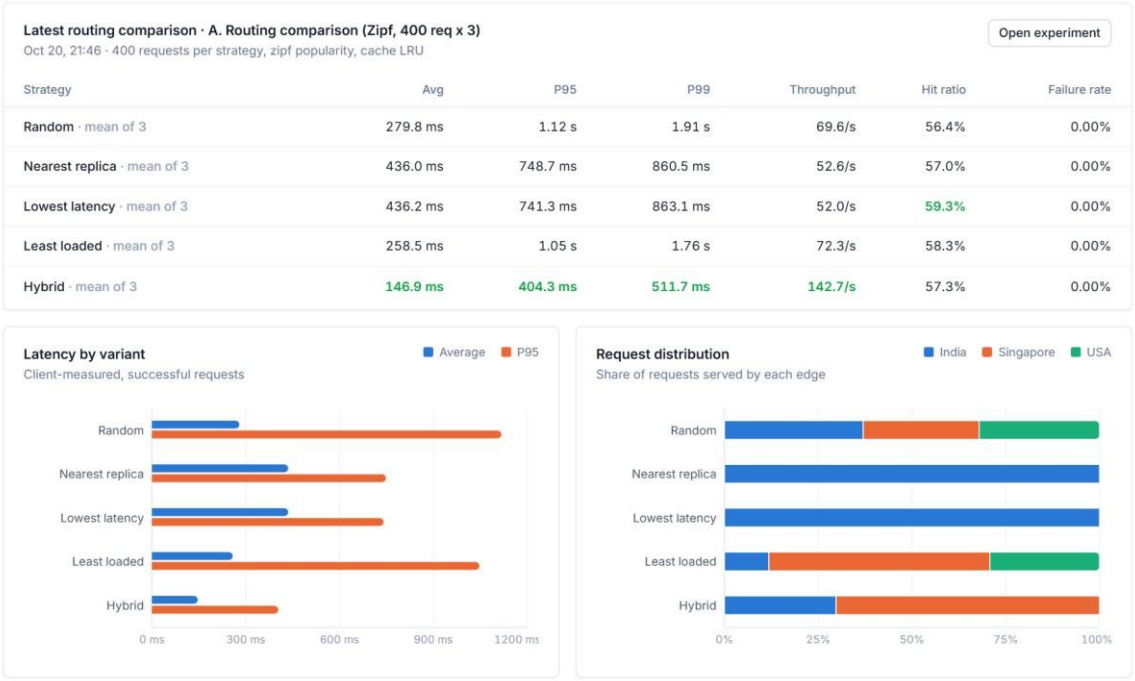


Figure B.1: Routing comparison results as displayed in the Routing Lab

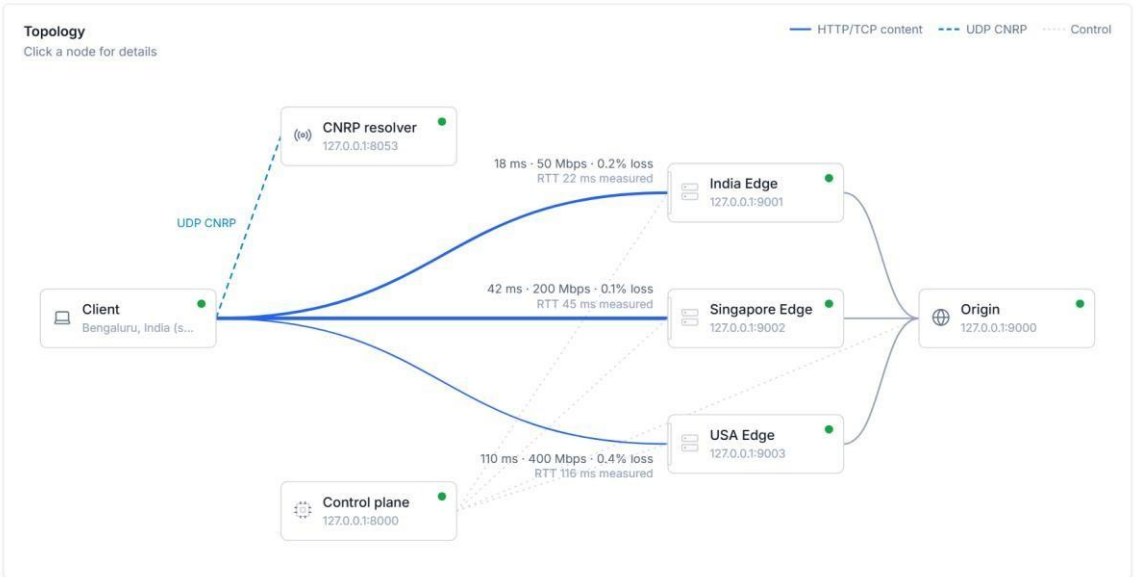


Figure B.2: Network Topology page (conceptual topology in Local Mode)

Raw results of all experiments (configurations, per-run summaries and timelines) are in `submission/evidence/experiment_results.json`; per-request records can be exported as CSV from the Experiments page. No packet captures were made.

Appendix C Individual Contributions

The project directory has no version-control history or task records from which individual contributions could be verified. The fields below are left for the team to complete. **Table C.1:** Individual contributions

Student Name and PRN	Work Completed	Evidence or Deliverable
More Geet Dattatraya (1262250006)	Overall Development	Working CDN simulator
Shinde Soham Manoj (1262250009)	Documentation	Word Report
Jagtap Prasad Sachin (1262250018)	Frontend & Caching Feature	UI running on local machine
Gaikwad Shreyash Sachin (1262250005)	Documentation	PPT